coding an technical writing

1. Project folder structure

```
CineBook/
├── Makefile
├── include/
│   ├── Models.h
│   └── Cinema.h
├── src/
│   ├── main.cpp
│   ├── Models.cpp
│   └── Cinema.cpp
├── public/
│   └── cinebook/
│       ├── index.html
│       ├── styles.css
│       └── app.js
├── tests/
│   └── test_cinema.cpp
└── data/
```

include/Models.h:

```
#pragma once


#include <string>
#include <vector>
```

```
class Movie {
private:
    int id_;
    std::string title_;
    std::string genre_;
    double price_;

public:
    Movie(int id, std::string title,
          std::string genre, double price);

    int getId() const;
    const std::string& getTitle() const;
    const std::string& getGenre() const;
    double getPrice() const;
};

class Seat {
private:
    std::string label_;
    bool booked_;

public:
    explicit Seat(std::string label, bool booked = false);
```

```
    const std::string& getLabel() const;
    bool isBooked() const;
    void setBooked(bool booked);
};

class Customer {
private:
    std::string name_;

public:
    explicit Customer(std::string name);

    const std::string& getName() const;
};

class Show {
private:
    int id_;
    int movieId_;
    std::string time_;
    std::vector<Seat> seats_;

public:
    Show(int id, int movieId, std::string time);

    int getId() const;
```

```
    int getMovieId() const;
    const std::string& getTime() const;
    const std::vector<Seat>& getSeats() const;
    bool hasSeat(const std::string& label) const;
};

class Ticket {
public:
    virtual ~Ticket() = default;

    virtual double calculatePrice(
        double basePrice, int quantity
    ) const = 0;

    virtual std::string type() const = 0;
};

class RegularTicket : public Ticket {
public:
    double calculatePrice(
        double basePrice, int quantity
    ) const override;

    std::string type() const override;
};
```

```
class Booking {
private:
    std::string id_;
    Customer customer_;
    int movieId_;
    int showId_;
    std::string movieTitle_;
    std::string showTime_;
    std::vector<std::string> seatLabels_;
    double total_;
    bool active_;

public:
    Booking(
        std::string id,
        Customer customer,
        int movieId,
        int showId,
        std::string movieTitle,
        std::string showTime,
        std::vector<std::string> seatLabels,
        double total,
        bool active = true
    );

    const std::string& getId() const;
```

```
    const Customer& getCustomer() const;
    int getMovieId() const;
    int getShowId() const;
    const std::string& getMovieTitle() const;
    const std::string& getShowTime() const;
    const std::vector<std::string>& getSeatLabels() const;
    double getTotal() const;
    bool isActive() const;

    void cancel();
};
```

3. include/Cinema.h:

```
#pragma once

#include "Models.h"

#include <stdexcept>
#include <string>
#include <vector>

class BookingException : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};
```

```
class Cinema {
private:
    std::vector<Movie> movies_;
    std::vector<Show> shows_;
    std::vector<Booking> bookings_;
    std::string dataFile_;

    const Movie& findMovie(int movieId) const;
    const Show& findShow(int showId) const;

    void saveBookings() const;
    void loadBookings();

    std::string makeBookingId() const;

public:
    explicit Cinema(
        std::string dataFile = "data/bookings.txt"
    );

    const std::vector<Movie>& getMovies() const;
    const std::vector<Show>& getShows() const;
    const std::vector<Booking>& getBookings() const;

    std::vector<std::string> getBookedSeats(int showId) const;
```

```
    Booking createBooking(
        const std::string& customerName,
        int movieId,
        int showId,
        const std::vector<std::string>& seats
    );

    void cancelBooking(const std::string& bookingId);

    std::string moviesJson() const;
    std::string seatsJson(int showId) const;
    std::string bookingsJson() const;

    static std::string bookingJson(const Booking& booking);
};
```

src/Models.cpp:

```
#include "Models.h"

#include <algorithm>
#include <utility>

Movie::Movie(
    int id,
    std::string title,
    std::string genre,
```

```
    double price
) : id_(id),
    title_(std::move(title)),
    genre_(std::move(genre)),
    price_(price) {}

int Movie::getId() const {
    return id_;
}

const std::string& Movie::getTitle() const {
    return title_;
}

const std::string& Movie::getGenre() const {
    return genre_;
}

double Movie::getPrice() const {
    return price_;
}

Seat::Seat(std::string label, bool booked)
    : label_(std::move(label)), booked_(booked) {}

const std::string& Seat::getLabel() const {
```

```
    return label_;
}

bool Seat::isBooked() const {
    return booked_;
}

void Seat::setBooked(bool booked) {
    booked_ = booked;
}

Customer::Customer(std::string name)
    : name_(std::move(name)) {}

const std::string& Customer::getName() const {
    return name_;
}

Show::Show(int id, int movieId, std::string time)
    : id_(id),
      movieId_(movieId),
      time_(std::move(time)) {

    for (char row = 'A'; row <= 'E'; ++row) {
        for (int number = 1; number <= 8; ++number) {
            seats_.emplace_back(
```

```
                std::string(1, row) + std::to_string(number)
            );
        }
    }
}

int Show::getId() const {
    return id_;
}

int Show::getMovieId() const {
    return movieId_;
}

const std::string& Show::getTime() const {
    return time_;
}

const std::vector<Seat>& Show::getSeats() const {
    return seats_;
}

bool Show::hasSeat(const std::string& label) const {
    return std::any_of(
        seats_.begin(),
        seats_.end(),
```

```
        [&](const Seat& seat) {
            return seat.getLabel() == label;
        }
    );
}

double RegularTicket::calculatePrice(
    double basePrice, int quantity
) const {
    return basePrice * quantity;
}

std::string RegularTicket::type() const {
    return "Regular";
}

Booking::Booking(
    std::string id,
    Customer customer,
    int movieId,
    int showId,
    std::string movieTitle,
    std::string showTime,
    std::vector<std::string> seatLabels,
    double total,
    bool active
```

```
) : id_(std::move(id)),
    customer_(std::move(customer)),
    movieId_(movieId),
    showId_(showId),
    movieTitle_(std::move(movieTitle)),
    showTime_(std::move(showTime)),
    seatLabels_(std::move(seatLabels)),
    total_(total),
    active_(active) {}

const std::string& Booking::getId() const {
    return id_;
}

const Customer& Booking::getCustomer() const {
    return customer_;
}

int Booking::getMovieId() const {
    return movieId_;
}

int Booking::getShowId() const {
    return showId_;
}
```

```
const std::string& Booking::getMovieTitle() const {
    return movieTitle_;
}

const std::string& Booking::getShowTime() const {
    return showTime_;
}

const std::vector<std::string>& Booking::getSeatLabels() const {
    return seatLabels_;
}

double Booking::getTotal() const {
    return total_;
}

bool Booking::isActive() const {
    return active_;
}

void Booking::cancel() {
    active_ = false;
}
```

5. src/Cinema.cpp

In this part of coding movie data, booking validation, price calculation, duplicate-seat prevention and the logic of the file are save.

```
#include "Cinema.h"

#include <algorithm>
#include <chrono>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <memory>
#include <set>
#include <sstream>
#include <utility>

namespace {

std::vector<std::string> split(
    const std::string& text, char delimiter
) {
    std::vector<std::string> parts;
    std::stringstream stream(text);
    std::string part;

    while (std::getline(stream, part, delimiter)) {
        parts.push_back(part);
```

```
    }

    return parts;
}

std::string join(
    const std::vector<std::string>& values,
    const std::string& separator
) {
    std::ostringstream out;

    for (std::size_t i = 0; i < values.size(); ++i) {
        if (i) out << separator;
        out << values[i];
    }

    return out.str();
}

std::string jsonEscape(const std::string& value) {
    std::ostringstream out;

    for (char c : value) {
        switch (c) {
            case '"': out << "\\\""; break;
            case '\\': out << "\\\\"; break;
```

```
            case '\n': out << "\\n"; break;
            case '\r': break;
            case '\t': out << "\\t"; break;
            default: out << c;
        }
    }

    return out.str();
}

std::string safeField(std::string value) {
    for (char& c : value) {
        if (c == '|' || c == '\n' || c == '\r') {
            c = ' ';
        }
    }

    return value;
}

}

Cinema::Cinema(std::string dataFile)
    : dataFile_(std::move(dataFile)) {

    movies_ = {
```

```
        Movie(1, "Midnight Protocol", "Sci-Fi / Thriller", 12.00),
        Movie(2, "The Last Horizon", "Adventure / Drama", 10.00),
        Movie(3, "Crimson Echo", "Mystery / Crime", 11.00),
        Movie(4, "Paper Moons", "Romance / Comedy", 9.00)
    };

    int showId = 1;

    const std::vector<std::string> times = {
        "12:30 PM", "4:15 PM", "8:00 PM"
    };

    for (const Movie& movie : movies_) {
        for (const std::string& time : times) {
            shows_.emplace_back(
                showId++, movie.getId(), time
            );
        }
    }

    loadBookings();
}

const std::vector<Movie>& Cinema::getMovies() const {
    return movies_;
}
```

```
const std::vector<Show>& Cinema::getShows() const {
    return shows_;
}

const std::vector<Booking>& Cinema::getBookings() const {
    return bookings_;
}

const Movie& Cinema::findMovie(int movieId) const {
    auto found = std::find_if(
        movies_.begin(), movies_.end(),
        [&](const Movie& movie) {
            return movie.getId() == movieId;
        }
    );

    if (found == movies_.end()) {
        throw BookingException("Movie not found.");
    }

    return *found;
}

const Show& Cinema::findShow(int showId) const {
    auto found = std::find_if(
```

```
        shows_.begin(), shows_.end(),
        [&](const Show& show) {
            return show.getId() == showId;
        }
    );

    if (found == shows_.end()) {
        throw BookingException("Showtime not found.");
    }

    return *found;
}

std::vector<std::string> Cinema::getBookedSeats(
    int showId
) const {
    findShow(showId);

    std::vector<std::string> result;

    for (const Booking& booking : bookings_) {
        if (booking.isActive() && booking.getShowId() == showId) {
            result.insert(
                result.end(),
                booking.getSeatLabels().begin(),
                booking.getSeatLabels().end()
```

```
            );
        }
    }

    return result;
}

Booking Cinema::createBooking(
    const std::string& customerName,
    int movieId,
    int showId,
    const std::vector<std::string>& seats
) {
    if (customerName.empty()) {
        throw BookingException("Customer name is required.");
    }

    if (seats.empty()) {
        throw BookingException("Select at least one seat.");
    }

    const Movie& movie = findMovie(movieId);
    const Show& show = findShow(showId);

    if (show.getMovieId() != movieId) {
        throw BookingException(
```

```
        "The showtime does not belong to this movie."
    );
}

std::set<std::string> uniqueSeats;

for (const std::string& seat : seats) {
    if (!show.hasSeat(seat)) {
        throw BookingException("Invalid seat: " + seat);
    }

    if (!uniqueSeats.insert(seat).second) {
        throw BookingException("Duplicate seat selected.");
    }
}

const auto booked = getBookedSeats(showId);

for (const std::string& seat : seats) {
    if (std::find(booked.begin(), booked.end(), seat)
        != booked.end()) {
        throw BookingException(
            "Seat " + seat + " is already booked."
        );
    }
}
```

```
std::unique_ptr<Ticket> ticket =
    std::make_unique<RegularTicket>();

const double total = ticket->calculatePrice(
    movie.getPrice(),
    static_cast<int>(seats.size())
);

const auto now = std::chrono::system_clock::now()
    .time_since_epoch();

const auto timestamp =
    std::chrono::duration_cast<std::chrono::milliseconds>(
        now
    ).count();

const std::string id =
    "CB" + std::to_string(timestamp)
    + std::to_string(bookings_.size() % 10);

bookings_.emplace_back(
    id,
    Customer(safeField(customerName)),
    movieId,
    showId,
```

```
        movie.getTitle(),
        show.getTime(),
        seats,
        total
    );

    saveBookings();

    return bookings_.back();
}

void Cinema::cancelBooking(const std::string& bookingId) {
    auto found = std::find_if(
        bookings_.begin(), bookings_.end(),
        [&](const Booking& booking) {
            return booking.getId() == bookingId;
        }
    );
  if (found == bookings_.end()) {
        throw BookingException("Booking ID not found.");
    }

    if (!found->isActive()) {
        throw BookingException("Booking is already cancelled.");
    }
```

```
    found->cancel();
    saveBookings();
}

void Cinema::loadBookings() {
    std::ifstream file(dataFile_);
    std::string line;

    while (std::getline(file, line)) {
        const auto fields = split(line, '|');

        if (fields.size() != 9) continue;

        try {
            bookings_.emplace_back(
                fields[0],
                Customer(fields[1]),
                std::stoi(fields[2]),
                std::stoi(fields[3]),
                fields[4],
                fields[5],
                split(fields[6], ','),
                std::stod(fields[7]),
                fields[8] == "1"
            );
        } catch (...) {
```

```
            // Ignore malformed records.
        }
    }
}

void Cinema::saveBookings() const {
    const std::filesystem::path path(dataFile_);

    if (path.has_parent_path()) {
        std::filesystem::create_directories(
            path.parent_path()
        );
    }

    std::ofstream file(dataFile_, std::ios::trunc);

    if (!file) {
        throw BookingException("Could not save booking data.");
    }

    for (const Booking& booking : bookings_) {
        file << safeField(booking.getId()) << '|'
             << safeField(booking.getCustomer().getName()) << '|'
             << booking.getMovieId() << '|'
             << booking.getShowId() << '|'
             << safeField(booking.getMovieTitle()) << '|'
```

```
            << safeField(booking.getShowTime()) << '|'
            << join(booking.getSeatLabels(), ",") << '|'
            << std::fixed << std::setprecision(2)
            << booking.getTotal() << '|'
            << (booking.isActive() ? "1" : "0")
            << '\n';
    }
}

std::string Cinema::moviesJson() const {
    std::ostringstream out;
    out << "{\"movies\":[";

    for (std::size_t i = 0; i < movies_.size(); ++i) {
        const Movie& movie = movies_[i];

        if (i) out << ',';

        out << "{\"id\":" << movie.getId()
            << ",\"title\":\"" << jsonEscape(movie.getTitle())
            << "\",\"genre\":\"" << jsonEscape(movie.getGenre())
            << "\",\"price\":" << movie.getPrice()
            << ",\"shows\":[";

        bool first = true;
```

```
        for (const Show& show : shows_) {
            if (show.getMovieId() != movie.getId()) continue;

            if (!first) out << ',';
            first = false;

            out << "{\"id\":" << show.getId()
                << ",\"time\":\"" << jsonEscape(show.getTime())
                << "\"}";
        }

        out << "]}";
    }

    out << "]}";
    return out.str();
}

std::string Cinema::seatsJson(int showId) const {
    const Show& show = findShow(showId);
    const auto booked = getBookedSeats(showId);

    std::ostringstream out;
    out << "{\"showId\":" << showId << ",\"seats\":[";

    for (std::size_t i = 0; i < show.getSeats().size(); ++i) {
```

```
        if (i) out << ',';

        const std::string& label =
            show.getSeats()[i].getLabel();

        const bool isBooked =
            std::find(booked.begin(), booked.end(), label)
            != booked.end();

        out << "{\"label\":\"" << label
            << "\",\"booked\":"
            << (isBooked ? "true" : "false") << "}";
    }

    out << "]}";
    return out.str();
}

std::string Cinema::bookingJson(const Booking& booking) {
    std::ostringstream out;

    out << "{\"id\":\"" << jsonEscape(booking.getId())
        << "\",\"customer\":\""
        << jsonEscape(booking.getCustomer().getName())
        << "\",\"movieId\":" << booking.getMovieId()
        << ",\"showId\":" << booking.getShowId()
```

```
        << ",\"movieTitle\":\""
        << jsonEscape(booking.getMovieTitle())
        << "\",\"showTime\":\""
        << jsonEscape(booking.getShowTime())
        << "\",\"seats\":[";

    for (std::size_t i = 0;
         i < booking.getSeatLabels().size(); ++i) {
        if (i) out << ',';

        out << "\""
            << jsonEscape(booking.getSeatLabels()[i])
            << "\"";
    }

    out << "],\"total\":" << booking.getTotal()
        << ",\"active\":"
        << (booking.isActive() ? "true" : "false")
        << "}";

    return out.str();
}

std::string Cinema::bookingsJson() const {
    std::ostringstream out;
    out << "{\"bookings\":[";
```

```
    for (std::size_t i = 0; i < bookings_.size(); ++i) {
        if (i) out << ',';

        out << bookingJson(
            bookings_[bookings_.size() - 1 - i]
        );
    }

    out << "]}";
    return out.str();
}
```

6. Makefile:

```
CXX := g++
CXXFLAGS := -std=c++20 -Wall -Wextra -pedantic -O2 -Iinclude

SOURCES := src/Models.cpp src/Cinema.cpp
TARGET := cinebook

.PHONY: all run test clean

all: $(TARGET)

$(TARGET): $(SOURCES) src/main.cpp
$(CXX) $(CXXFLAGS) $(SOURCES) src/main.cpp -o $(TARGET)
```

```
run: $(TARGET)
./$(TARGET)

test: $(SOURCES) tests/test_cinema.cpp
$(CXX) $(CXXFLAGS) $(SOURCES) tests/test_cinema.cpp -o
cinebook_tests
./cinebook_tests

clean:
rm -f $(TARGET) cinebook_tests
```